Automated Web Scanner

A Python Tool for Auditing Security Headers and CORS Misconfigurations

Project Report

Submitted by: **Mohit Tiwari**

Roll No / Enrollment No: **[Your Roll No]**

Course: **B. Tech CSE**

University: **Graphic Era Hill University, Haldwani**

Guide: **[Guide Name]**

Date: **October 2026**

Abstract

Web applications rely on HTTP response headers and Cross-Origin Resource Sharing (CORS) policies to defend against attacks such as Cross-Site Scripting (XSS), clickjacking, protocol downgrade and cross-origin data theft. Misconfiguration of these controls is common and is listed under 'Security Misconfiguration' in the OWASP Top 10. This project presents the **Automated Web Scanner**, a command-line tool written in Python that audits one or many websites for missing or weak security headers, insecure cookie flags, missing HTTPS redirection and dangerous CORS configurations. Each finding is assigned a severity level and a remediation tip, and every target receives an overall score and grade. Results can be exported as JSON for further analysis. The tool performs only passive, non-intrusive HTTP requests and is intended for use on systems the tester owns or is authorised to assess.

Table of Contents

1. Introduction

2. Problem Statement and Objectives

3. Scope and Ethical Considerations

4. Technologies Used

5. System Design

6. Implementation Details

7. Usage and Sample Output

8. Testing

9. Limitations

10. Future Scope

11. Conclusion

12. References

1. Introduction

Every time a browser loads a web page, the server replies with HTTP headers that instruct the browser how to behave. Security-related headers such as *Content-Security-Policy* and *Strict-Transport-Security* enable built-in browser defences at almost no cost. When they are absent or weak, otherwise secure applications become easier to exploit.

CORS is another critical mechanism. By default, browsers enforce the Same-Origin Policy, which stops a page on one origin from reading responses from another. CORS relaxes this policy in a controlled way. If a server reflects any *Origin* header it receives, or trusts the 'null' origin while also allowing credentials, a malicious website can read sensitive data on behalf of a logged-in victim.

Manually checking these settings for many sites is slow and error-prone. This project automates the process and produces a clear, prioritised report.

2. Problem Statement and Objectives

Problem statement: Develop an automated tool that audits websites for missing or misconfigured security headers and CORS flaws, and reports the findings with severity and remediation guidance.

Objectives:

- Detect missing and weak HTTP security headers.
- Identify information-leaking headers such as Server and X-Powered-By.
- Test CORS behaviour using multiple crafted Origin values and a preflight request.
- Audit cookie attributes (Secure, HttpOnly, SameSite) and HTTP to HTTPS redirection.
- Score each site and export results in a machine-readable format.

3. Scope and Ethical Considerations

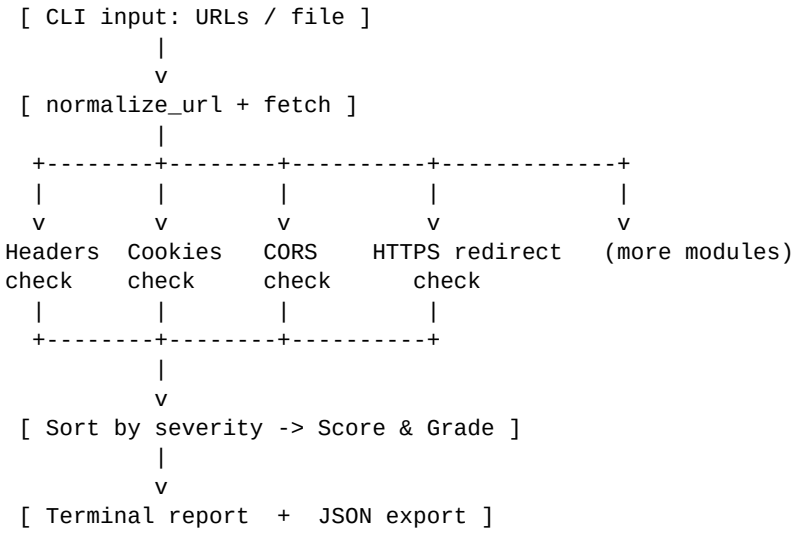
The scanner sends ordinary GET and OPTIONS requests and inspects the responses. It does not attempt exploitation, brute force or denial of service. Scanning systems without permission may be illegal; therefore the tool prints a warning at start-up and should only be used on websites owned by the user, on dedicated practice targets, or where written authorisation exists.

4. Technologies Used

Component	Details
Language	Python 3.8+
HTTP library	requests
Standard modules	argparse, json, urllib.parse, datetime, sys
Output	Colour-coded terminal report and JSON file

5. System Design

The scanner follows a simple modular pipeline. A target URL is normalised, fetched once, and the response is passed to independent check modules. Findings from all modules are merged, sorted by severity, scored and printed.



5.1 Modules

Module	Function	Purpose
check_security_headers	Verifies presence and value quality of six key headers; flags leaky headers	Browser-side protection
check_cookies	Inspects Secure, HttpOnly and SameSite flags	Session safety
check_cors	Sends four spoofed Origins and one preflight request	Cross-origin trust
check_https_redirect	Requests the HTTP version and expects a redirect to HTTPS	Transport security
grade	Computes score out of 100 and letter grade	Summary

6. Implementation Details

6.1 Security Header Audit

Header	Severity	Protects against
Strict-Transport-Security	HIGH	Protocol downgrade, SSL stripping
Content-Security-Policy	HIGH	XSS and data injection
X-Frame-Options	MEDIUM	Clickjacking
X-Content-Type-Options	MEDIUM	MIME sniffing
Referrer-Policy	LOW	URL leakage to third parties
Permissions-Policy	LOW	Abuse of browser features

Beyond presence checks, the tool flags an HSTS max-age below six months, a CSP containing 'unsafe-inline' or 'unsafe-eval', and headers such as Server or X-Powered-By that reveal software versions.

6.2 CORS Testing

The scanner sends the same request with different Origin values and examines *Access-Control-Allow-Origin* (ACAO) and *Access-Control-Allow-Credentials* (ACAC) in the reply:

- **Arbitrary origin:** an unrelated attacker domain. If reflected with credentials enabled, the finding is CRITICAL.
- **Null origin:** trusted 'null' can be abused through sandboxed iframes.
- **Subdomain spoof:** target host with an attacker suffix, detecting weak suffix matching.
- **Prefix spoof:** attacker prefix added to the host, detecting weak prefix matching.
- **Preflight:** an OPTIONS request checking whether risky methods (DELETE, PUT, PATCH) are allowed to untrusted origins.

6.3 Scoring

Each target starts at 100 points. Penalties are subtracted per finding and the remainder is mapped to a grade.

Severity	Penalty		Score	Grade
CRITICAL	30		90 - 100	A
HIGH	15		75 - 89	B
MEDIUM	8		60 - 74	C
LOW	3		40 - 59	D
INFO	0		below 40	F

7. Usage and Sample Output

7.1 Installation and Commands

```
pip install -r requirements.txt

python scanner.py https://example.com
python scanner.py site1.com site2.com
python scanner.py -f targets.txt -o report.json
```

7.2 Sample Output (illustrative)

```
========================================================================
Target : https://example.com
Status : 200   Score: 61/100   Grade: C
------------------------------------------------------------------------
[HIGH] Content-Security-Policy
   Issue: CSP missing: no defence-in-depth against XSS.
   Fix  : Define a restrictive CSP, e.g. default-src 'self'
[MEDIUM] X-Frame-Options
   Issue: Clickjacking protection missing.
   Fix  : Add: X-Frame-Options: DENY (or CSP frame-ancestors)
[LOW] Referrer-Policy
   Issue: Referrer policy missing.
   Fix  : Add: Referrer-Policy: strict-origin-when-cross-origin
```

8. Testing

The tool should be validated against a local test server with deliberately chosen header and CORS settings, and against sites that are intentionally vulnerable for practice. Expected behaviour is summarised below.

Test case	Expected result
Server with no security headers	All six missing-header findings, low score
Server reflecting any Origin with credentials	CRITICAL CORS finding
Server allowing ACAO: null	HIGH CORS finding
HSTS max-age=300	MEDIUM weak-HSTS finding
Cookie without Secure/HttpOnly	MEDIUM cookie finding
Unreachable host	Error message, scan continues with next target

9. Limitations

- Scans only the URL given, not every page or API route of a site.
- CORS behaviour can differ per endpoint; results reflect the tested URL only.
- Cannot judge whether a CSP is logically strong, only whether obvious weaknesses exist.
- Sequential requests; large target lists are slower than an asynchronous design.

10. Future Scope

- Asynchronous scanning for faster bulk audits.
- HTML and PDF report generation with charts.
- Crawling to test multiple endpoints per site.
- TLS certificate and cipher analysis.
- Scheduled scans with email alerts and historical comparison.

11. Conclusion

The Automated Web Scanner demonstrates how common web security misconfigurations can be detected quickly and consistently with a small Python program. By combining header analysis, cookie checks, HTTPS verification and multi-origin CORS testing, it gives developers a prioritised list of fixes and a simple score to track improvement. The project also builds practical understanding of HTTP security mechanisms, the Same-Origin Policy and responsible security testing.

12. References

- OWASP Foundation, OWASP Top 10 - Security Misconfiguration. https://owasp.org
- OWASP Secure Headers Project. https://owasp.org/www-project-secure-headers/
- MDN Web Docs, HTTP Headers and CORS. https://developer.mozilla.org
- RFC 6797 - HTTP Strict Transport Security (HSTS).
- RFC 6454 - The Web Origin Concept.
- Python Requests documentation. https://requests.readthedocs.io